Reading Sign Language

Stage 1: Raw pixels

Mohamed Shawki Aia Khawaled

November 13, 2026

1 Self-check

1.1 Weight vector of one class as a 28×28 image

A pixel receiving a large positive weight means that for the class, that pixel is often bright, and less bright for the other classes. Therefore multiplying the weight by the scaled pixel value raises the score, which raises the probability for that class to be the right class for that input in relation to the other classes. A large negative weight means that, for this class, the pixel is usually dark, so if it's bright in an input, the score drops. When the weight vector is reshaped to 28×28 and shown as an image, the positive weights roughly trace where this letter's hand usually appears, and the negative weights mark the areas that should stay empty for this letter.

1.2 Why a 2-pixel shift is a very different input

Because the weight map is not shifted along with the image, each weight stays fixed on a specific pixel, and it doesn't care about its relation to the neighboring pixels. 2 pixels is a lot in relation to the 28 pixel per dimension, which could lead to moving most of a finger off of the pixels it used to cover. After the shift, the bright parts of the hand fall on pixels with different weights, causing parts of the hand that used to be on large positive weights to land on small or negative weights, which changes the weighted sum and the scores of every letter, so the input looks very different to the classifier.

1.3 Random validation split versus the official test set

Because the validation set consists of altered, near-copy photos from those 1700 photos, whose other copies the model has already observed and trained off of, and that inflates the accuracy and gives an overly optimistic result, whereas an official test set would contain photos that are truly unseen to the model.

2 Data exploration

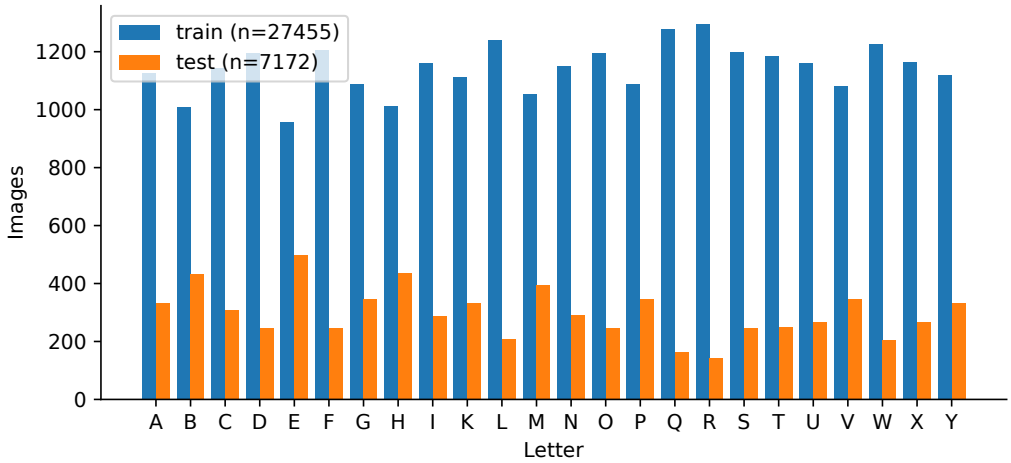
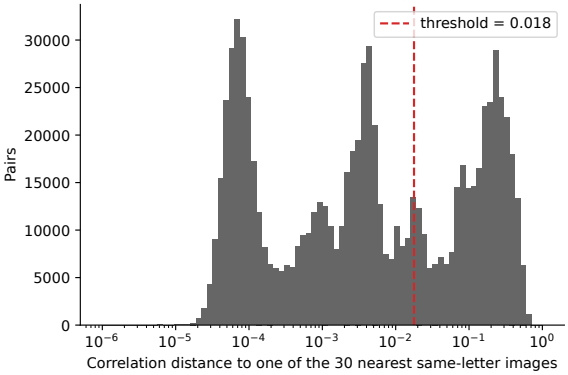


Figure 1: Number of images per letter in the training and test sets.



Figure 2: Five training examples of each letter.



(a) Nearest-neighbour distances between training images.



(b) One group of near-duplicate training images.

Figure 3: Evidence for near-duplicate images in the training set and the resulting group-aware validation split.

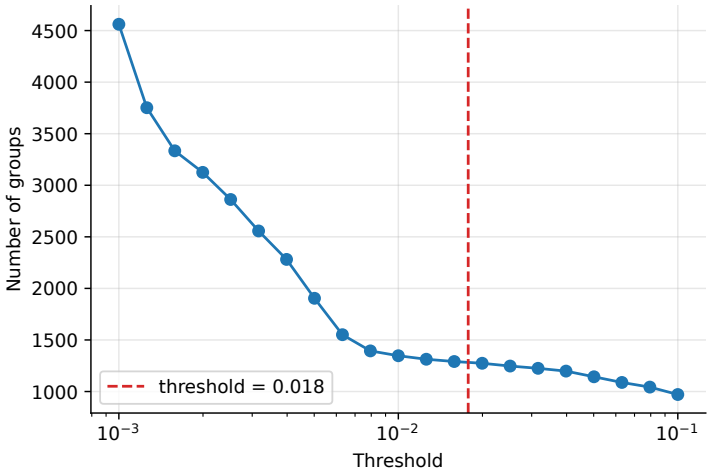


Figure 4: Number of near-duplicate groups as a function of the distance threshold. The dashed line marks the chosen threshold.

We expect these pairs to be confused: G and T, E and S, D and R. These letter pairs have similar hand shapes and gestures, so the classifier will probably confuse them.

3 Results

Here n is the number of images, $\hat{y}_i$ the predicted letter and y_i the true letter of image i. TP_c is the number of images of letter c predicted as c, FP_c the number of images of other letters predicted as c, and FN_c the number of images of c predicted as another letter.

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}(\hat{y}_i = y_i)$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

$$F_{1,c} = \frac{2\,\text{Precision}_c\,\text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \qquad \text{Macro-}F_1 = \frac{1}{24}\sum_{c=1}^{24} F_{1,c}$$

Table 1: Validation accuracy for several values of C on the group-aware split and on a plain random split, with training time.

C	Val acc (group split)	Val acc (random split)	Fit time (s)	Converged
0.001	0.426	0.618	12.1	yes
0.01	0.586	0.832	20.2	yes
0.1	0.658	0.983	35.3	yes
1	0.656	0.999	22.8	yes
10	0.656	1.000	19.7	yes
100	0.623	0.998	14.1	yes

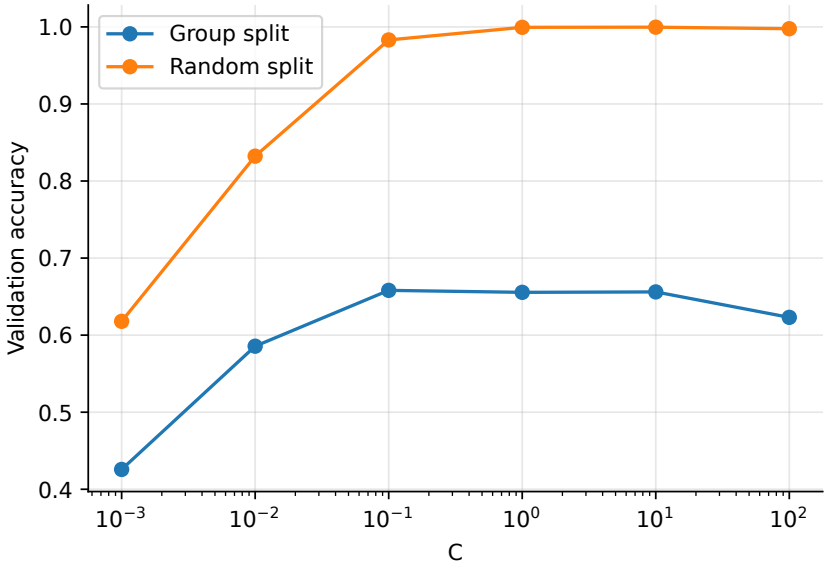


Figure 5: Validation accuracy as a function of C for the two splits.

Table 2: Accuracy and macro-F1 of the chosen model on the validation splits and on the official test set.

Evaluated on	Accuracy	Macro-F1
Validation (group split)	0.658	0.662
Validation (random split)	0.983	0.983
Test	0.660	0.645

$C = 0.1$ was chosen because it gave the highest validation accuracy between $C = 0.001, \ldots, 100$ for the group split (0.658), which is the rule from our plan. The F1 score happens to agree, since it is also highest at $C = 0.1$ (0.662). The validation accuracy for the group split rose between $C = 0.001$ to $C = 0.1$, then stayed almost flat from $C = 0.1$ to $C = 10$ (0.658, 0.656, 0.656), and only dropped at $C = 100$ (0.623), but stayed higher than the values from $C = 0.001$ and $C = 0.01$ (0.426, 0.586).

In random split accuracy, we can see that it is much higher than group split accuracy, scoring approximately 0.9996 for $C = 10$, which is the maximum for all C. Because in random split, the validation group contains copies of the same images trained on the softmax classifier, because the training images were made from about 1700 photos with small changes. We checked in Stage 0 and discovered that they come from 1,281 photos. And if you randomly split them and then try to validate the classifier, it will classify them correctly, because it trained on the images before

with a slight change that won't affect its classification.

It turned out that there isn't a gap between test and validation accuracy in the group split (0.658 validation, 0.660 test). The validation group split gives a good estimate of the test accuracy. The validation set helped us choose $C = 0.1$, and the test set was used only once for the chosen C.

4 Weight images

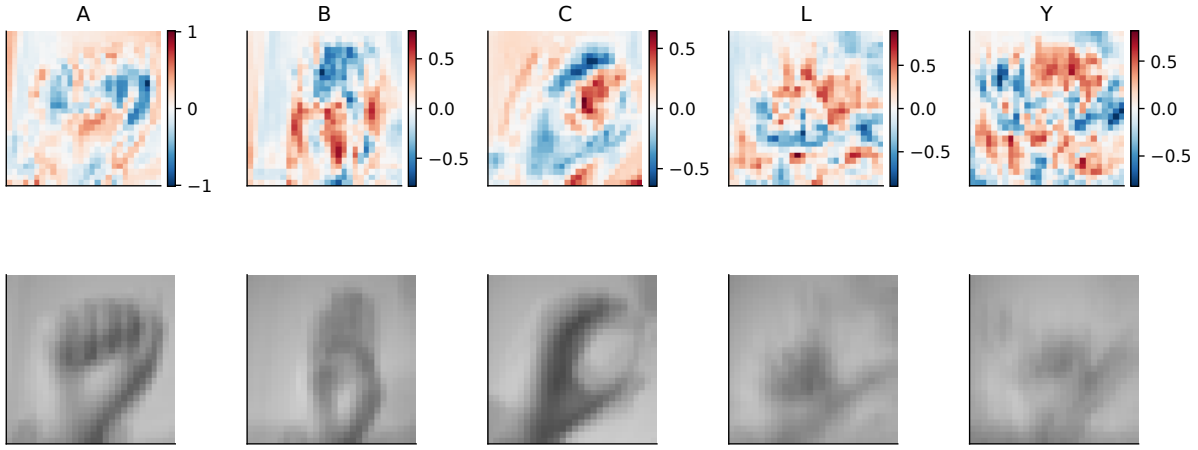


Figure 6: Top row: weight vectors of five letters reshaped to 28×28 (red positive, blue negative). Bottom row: mean training image of the same letters.

The weight images for A, B, C, L, Y show heat maps with blue color for negative weights, red color for positive weights, and white for almost zero. This clearly matches our interpretation for self-check question one. For regions of a letter image that are darker than in the other letters, the weights are negative (blue in the weight image), and for regions that are brighter than in the other letters, the weights are positive (red in the weight image). We can see that in each letter's average image, each region that is brighter or darker than in the other letters is connected to the red or blue region in the letter's weight image. The weight images aren't smooth, they are blurred and speckled. We can detect the blue and red regions and connect them to the dark and bright spots of the letter's average image for letters A, B, and especially C, which shows a blue region on the dark fingers and red on the bright gap the hand makes in the middle.

5 Confusions

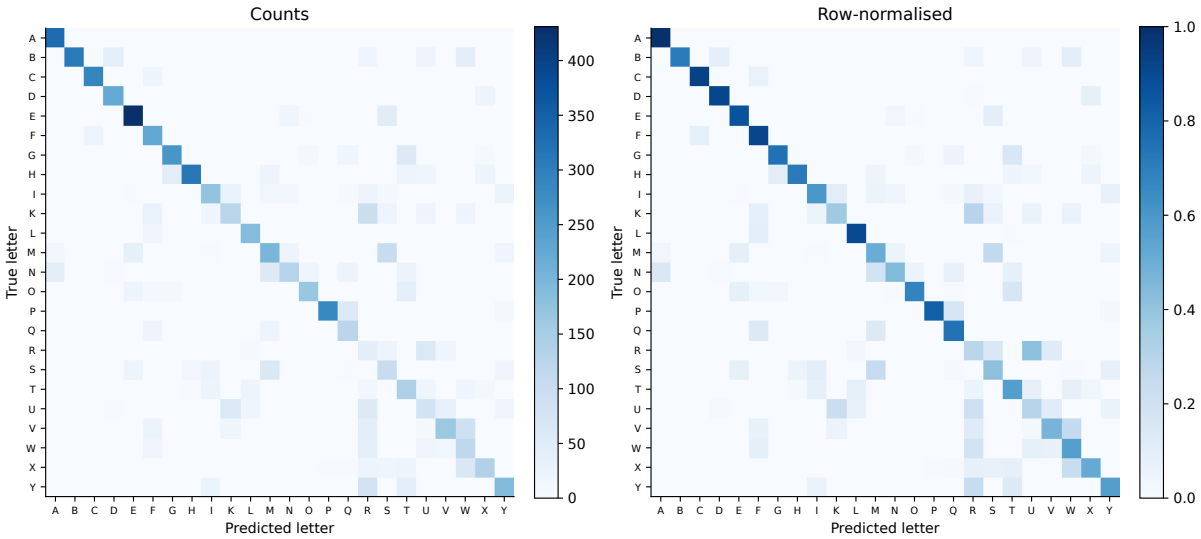


Figure 7: Confusion matrix on the test set: counts (left) and row-normalised (right).

Table 3: The five most confused letter pairs on the test set.

True	Predicted	Count	Reverse count	Total
M	S	103	61	164
R	U	60	55	115
V	W	89	16	105
K	R	97	0	97
U	K	60	21	81



Figure 8: For each confused pair: test images of the true letter that were misclassified (left four) and training images of the predicted letter (right four).

The most confused pairs are M–S, R–U, V–W, K–R and U–K.

A linear classifier on raw pixels can't separate them because the difference between the two confused letters are small details, like a one-finger difference or a small change in gesture, and sometimes the hands in both letters sit higher, lower, or tilted, and the distinguishing detail lands on different pixels each time. Because of that, a fixed weight per pixel can't follow it, especially since those changes are also found in the training images of the same letter. Then what happens is: suppose test image α, which is an S letter image, and the hand's position in α is shifted. Then α's bright pixels move onto pixels where M has positive (red) weights and S doesn't, so M's red region will sit on α's bright region. Then $z_M > z_S$, so P_M for α is larger than P_S, and α is labelled M.

We should note that K–R is one-sided, so only K is misclassified as R, because R has 1171 images in the group split training part, while K has only 928. So R weights might be larger, and indeed R has the largest weight vector of all 24 letters (length 8.14, while most are 6–7.7). R's intercept is not larger (-0.16, against K's 0.00), so the pull comes from the weights, not the bias. So it might attract similar letters like K: R is predicted 426 times on the test set, but only 144 test images are actually R. Also, there are only 144 R test images against 331 K, so R has fewer chances to be mistaken for K. In the confusion matrix, we can see that the highest mistake for R is U, not K (60 of the 144 R images became U, only 41 were correct, and 0 became K). Those confusions happen because the test images come from different photos than the training images.

6 Prediction vs. result

6.1 Question 1: how well does it do?

Prediction. Validation accuracy will be close to the test accuracy, both between 60% and 75%, because we made sure the validation set is not made using a random split. We predict that choosing a medium value for C ($C = 1$) would probably give the best result overall, since it gives

the best trade-off between bias and variance: it makes sure we are not overfitting, as with a high value of C, and that the weights are not close to zero, as with a small C.

Result. Accuracy is right (validation 0.658 and test 0.660, both inside the 60–75% we predicted) and C is 0.1 instead of 1, so we have close guesses. $C = 1$ got 0.6555, almost the same as $C = 0.1$ (0.6581), so it was a close call.

6.2 Question 2: what does it look at?

Prediction. If validation picks a high value of C, we expect the weight images to look like noisy speckles, because a weak penalty lets neighboring pixels get very different weights. With a smaller (medium) C, we expect a blurry outline, because the stronger penalty spreads the weight across neighboring pixels that carry the same information, giving smoother images.

Result. Yes, they look blurred and speckled as we predicted, but note that we only proved our prediction for the middle C, because we showed only the weight images for $C = 0.1$.

6.3 Question 3: which letters are confused?

Prediction. We predict that the pairs we suggested in the Data exploration section will be confused, because these letters will get similar weights. A linear classifier on raw pixels won't classify them correctly, because its only features are the raw pixels and the pixels of these letters are similar. Other classifiers, like a CNN, can learn features that help them find more distinct features for the classes. PCA finds the greatest variance across all images, and then the softmax classifier will learn from the new features, which can help because we dropped all the components with the least variance.

Result. Partly. The actual top-5 pairs are M–S (164), R–U (115), V–W (105), K–R (97) and U–K (81). Of our predicted pairs, E and S were confused (light blue color in the confusion matrix), and G and T, so these two are partly right. D and R were almost never confused, so that one is wrong. The PCA part of our prediction belongs to Stage 2, and we will check it there.

7 AI-use note

The AI wrote the Stage 1 code: the code in `src/` (including `softmax.py`), the Python notebook (`.ipynb`), and the code that makes the figures and tables. It also wrote `setup.bat` (including the LaTeX install), set up the LaTeX report structure including the figure and table captions, gave the metric formulas, explained concepts, gave hints, and checked our drafts for errors. We wrote all the answers, predictions and analysis ourselves, including the self-check questions and the report, and we ran the notebook ourselves.